

Computer Vision Exercises

Student Name - Student ID

1 Canny Edge Detector

The Canny algorithm is a standard, multi-stage method for accurately extracting image edges. In this exercise, you need to write a function that implements this algorithm, receives an input image, and applies the following five basic steps to it.

Input Image

The input image required for testing your code is provided. Please download the image first and test your code on it.

Important Note

The process of analyzing structure and extracting edges is performed exclusively on the intensity channel. Therefore, you must convert the color input image to grayscale as the first step.

Algorithm Steps

1. Gaussian Smoothing

Reduce noise by smoothing the image using a Gaussian filter with parameter σ .

2. Gradient Calculation

Using derivative filters (e.g., Sobel), compute:

$$|G| = \sqrt{G_x^2 + G_y^2} \quad (1)$$

$$\theta = \arctan\left(\frac{|G_y|}{|G_x|}\right) \quad (2)$$

3. Non-Maximum Suppression

Each candidate pixel is examined along its gradient direction. Pixel $f(p)$ is compared with its two neighbors a_1 and a_2 . If the condition $f(p) \geq a_1 \wedge f(p) > a_2$ is not met, the candidate is removed (set to zero).

4. Double Thresholding

Using high threshold TH_h and low threshold TH_l :

- If gradient $> TH_h$: strong edge
- If gradient $< TH_l$: not an edge (removed)
- If $TH_l \leq \text{gradient} \leq TH_h$: weak edge

5. Edge Tracking by Hysteresis

Strong edges are accepted as final. Weak edges are kept only if connected to a strong edge in their neighborhood.

Function Specifications

Implement `canny_edge_detection(image, sigma, low_threshold, high_threshold)` where:

- `image`: 2D numpy array (grayscale, 0-255)
- `sigma`: float (Gaussian standard deviation)
- `low_threshold`: float
- `high_threshold`: float

Allowed Libraries

Using standard libraries and ready-made functions like `cv2.Canny` and `cv2.GaussianBlur` is allowed.

Submission

Zip file named `IP_StudentName_StudentId` containing input image, output image, and report.

2 Image Sharpening: High-Boost Filter Combined with Sobel

Part 1: Implementation

Implement `advanced_high_boost_sobel(image, A=1.5, ksize=3, alpha=0.6)` performing:

1. Normalization to $[0, 1]$
2. Gaussian blur
3. High-Boost: $HB = A \cdot f(x, y) - f_{blur}(x, y)$
4. Sobel edge map: $S = \sqrt{G_x^2 + G_y^2}$
5. Final combination: $Output = (1 - \alpha) \cdot HB + \alpha \cdot S$

Display and save all intermediate images.

Part 2: Analysis Questions

1. Why does combining High-Boost with Sobel usually give better results than High-Boost alone?
2. How does changing α affect the output (especially $\alpha = 0$, $\alpha = 1$, and very high values)?
3. From a frequency perspective, what information does Sobel extract? What causes oversharpening or halos?

4. If the image has Gaussian noise, what problem does this method create? Suggest a practical solution.

Submission

Zip file named `IP_StudentName_StudentId` containing code, PDF with answers and screenshots, and output images.

3 Hough Transform

Combining Canny edge detection with Hough Transform is a classic method for detecting lines. First, Canny extracts edge pixels, then Hough Transform maps them to parameter space.

Requirements

1. Apply Canny, then Hough Transform. Tune parameters to extract main lines (not noise or textures).
2. Draw detected lines.
3. Create a 4-subplot display:
 - Canny binary output
 - Hough parameter space (accumulator matrix)
 - Detected lines on black background
 - Final output with lines drawn on original image
4. Analyze: How does changing the accumulator threshold affect output? How do improperly set Canny thresholds affect Hough space and detected lines?

Submission

Zip file named `IP_StudentName_StudentId` containing notebook (.ipynb with outputs and markdown explanations) and separate output images.